Fraud Analytics Assignment 2: Spectral Clustering

Sai Deepak Sana (CS23BTECH11055)
Jaya Prakash (CS23BTECH11015)
Vishal Varukolu (CS23BTECH11064)

April 7th, 2026

Contents

1 Introduction

Graph clustering is the task of partitioning nodes into groups such that nodes within a group are more strongly connected to each other than to nodes in other groups. It is widely used in community detection, network analysis, and fraud analytics.

We applied **spectral clustering** to an undirected graph with 600 nodes and 6,734 edges. The goals are:

1. determine the number of clusters k from the graph structure,
2. assign a cluster label to every node,
3. and report the final clustering clearly and reproducibly.

Spectral clustering is suitable here because it uses the smallest eigenvectors of the graph Laplacian to build a spectral embedding that captures global connectivity patterns better than purely local methods.

2 Dataset Description

The input file is an edge list CSV (`spectral_graph_600nodes_edges.csv`). Each row contains one undirected edge between two node IDs.

Table 1: Dataset statistics

Property	**Value**
Number of nodes	600
Number of edges	6,734
Node IDs	integers $0, 1, \ldots, 599$
Graph type	Undirected, unweighted
Connected?	Yes (1 connected component)
Min degree	9
Max degree	37
Mean degree	22.45

Assumption on node numbering. The implementation assumes that node IDs are exactly the contiguous range $\{0, 1, \ldots, 599\}$. After loading the graph, we explicitly add all nodes using `G.add_nodes_from(range(600))`. This ensures isolated nodes, if any, are included and also makes the matrix indexing consistent.

3 Process

3.1 Spectral Clustering Pipeline

The clustering procedure follows these steps:

1. Build the undirected graph from the edge list.
2. Compute the graph Laplacian.
3. Estimate the number of clusters using the eigengap heuristic.
4. Build a spectral embedding from the leading eigenvectors.
5. Run K-Means on the embedding.
6. Convert the final labels to 1-based indexing for reporting.

3.2 Graph Laplacian

Let $A \in \{0,1\}^{n \times n}$ be the adjacency matrix and D be the diagonal degree matrix. The **unnormalized Laplacian** is

$$L = D - A.$$

From the spectral clustering slides, two Laplacian variants are presented: the unnormalized Laplacian, which corresponds to the RatioCut formulation, and the normalized Laplacian, which is used for Balanced Cuts. For this dataset, the degree range is moderate (9 to 37), and the graph is already fairly balanced, so the unnormalized Laplacian is a reasonable choice. In this graph it produces a clear spectral structure and leads to stable clustering.

3.3 Eigengap Heuristic

The eigenvalues of L, sorted in ascending order,

$$\lambda_0 \leq \lambda_1 \leq \cdots \leq \lambda_{n-1},$$

provide information about graph partitioning. For a graph with k disconnected components, the first k eigenvalues are exactly zero. For a graph with k nearly disconnected communities, the first k eigenvalues are small and a noticeable gap appears after them.

We use the eigengap heuristic:

1. compute the smallest eigenvalues of L,
2. sort them in ascending order,
3. inspect the gaps $\lambda_{i+1} - \lambda_i$,
4. choose k just before the largest gap.

This is the practical way we apply the slide idea of using the eigenvectors corresponding to the smallest eigenvalues for spectral clustering.

3.4 Spectral Embedding and K-Means

After selecting k, we use the first k eigenvectors as a low-dimensional embedding:

$$U \in \mathbb{R}^{n \times k}.$$

Each row of U represents one node. We then row-normalize the embedding and apply K-Means to the normalized vectors.

Row normalization helps remove scale differences between eigenvectors and improves clustering stability.

3.5 Implementation Details

- **Language and libraries:** Python 3, NumPy, Pandas, NetworkX, SciPy, scikit-learn, Matplotlib.
- **Eigendecomposition:** `scipy.sparse.linalg.eigsh` with `which="SM"`.
- **Number of eigenvalues computed:** 20 smallest eigenvalues.
- **K-Means settings:** `n_init=20`, `max_iter=500`, `random_state=42`.
- **Reproducibility:** `np.random.seed(42)` is set.

```
A = csr_matrix((data, (rows, cols)), shape=(n, n))
degrees = np.array(A.sum(axis=1)).flatten()
D = csr_matrix((degrees, (range(n), range(n))), shape=(n, n))
L = D - A
```

Listing 1: Constructing the sparse Laplacian

```
eigenvalues, eigenvectors = eigsh(L, k=20, which='SM')
idx = np.argsort(eigenvalues)
eigenvalues, eigenvectors = eigenvalues[idx], eigenvectors[:, idx]

gaps = np.diff(eigenvalues)
k = int(np.argmax(gaps[1:]) + 2)   # skip the trivial first gap

U = eigenvectors[:, :k]
U_norm = normalize(U, norm='l2', axis=1)
```

Listing 2: Eigenvalues, eigengap, and embedding

```
kmeans = KMeans(n_clusters=k, n_init=20, max_iter=500, random_state=42)
kmeans.fit(U_norm)
cluster_labels = kmeans.labels_ + 1   # convert to 1-based labels
```

Listing 3: K-Means on the spectral embedding

4 Experiments and Results

4.1 Graph Construction

The graph was built successfully from the edge list. The notebook confirms:

- 600 nodes,
- 6,734 edges,
- one connected component,
- minimum degree 9,
- maximum degree 37,
- mean degree 22.45.

4.2 Eigenvalue Spectrum

We computed the 20 smallest eigenvalues of the Laplacian. The values are shown below.

Table 2: 20 smallest eigenvalues of the Laplacian

Index	Eigenvalue	Index	Eigenvalue
λ_0	0.000000	λ_{10}	11.101343
λ_1	4.797589	λ_{11}	11.223003
λ_2	5.001074	λ_{12}	11.326968
λ_3	5.155374	λ_{13}	11.495509
λ_4	8.340117	λ_{14}	11.606218
λ_5	10.057923	λ_{15}	11.712617
λ_6	10.115592	λ_{16}	11.766901
λ_7	10.644563	λ_{17}	11.812586
λ_8	10.779221	λ_{18}	11.855025
λ_9	10.999757	λ_{19}	11.945061

4.3 Eigengap Analysis

The consecutive eigengaps are:

Table 3: Eigengaps between consecutive eigenvalues

Gap	**Value**	**Comment**
$\lambda_1 - \lambda_0$	4.797589	trivial gap caused by $\lambda_0 = 0$
$\lambda_2 - \lambda_1$	0.203485	
$\lambda_3 - \lambda_2$	0.154300	
$\lambda_4 - \lambda_3$	3.184743	largest non-trivial gap
$\lambda_5 - \lambda_4$	1.717806	
$\lambda_6 - \lambda_5$	0.057669	
$\lambda_7 - \lambda_6$	0.528971	
$\lambda_8 - \lambda_7$	0.134659	
$\lambda_9 - \lambda_8$	0.220536	
$\lambda_{10} - \lambda_9$	0.101586	
$\lambda_{11} - \lambda_{10}$	0.121660	
$\lambda_{12} - \lambda_{11}$	0.103965	
$\lambda_{13} - \lambda_{12}$	0.168540	
$\lambda_{14} - \lambda_{13}$	0.110709	
$\lambda_{15} - \lambda_{14}$	0.106399	

The largest meaningful gap is between $\lambda_4 \approx 8.340117$ and $\lambda_5 \approx 10.057923$, with size 3.184743. By the eigengap heuristic, this gives:

$$k = 4.$$

4.4 Clustering Results

The embedding produced a clean K-Means partition.

Table 4: Cluster summary

Cluster ID	**Number of Nodes**
1	150
2	150
3	150
4	150
Total	**600**

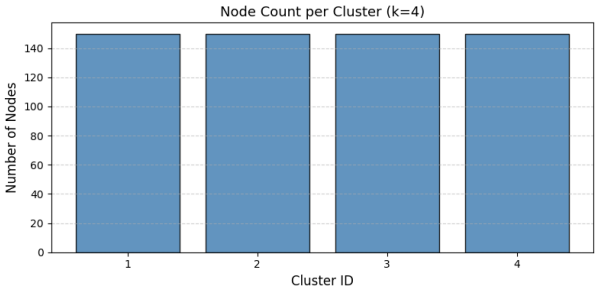

Figure 1: Number of nodes in each cluster. All clusters contain exactly 150 nodes, indicating a balanced partition.

K-Means converged in 2 iterations with final inertia 11.7827. All four clusters contain exactly 150 nodes, so the final partition is perfectly balanced.

Label convention. K-Means internally returns labels starting from 0. For the report and the CSV file, the labels are shifted to 1-based indexing, so the final labels are $\{1, 2, 3, 4\}$.

4.5 Visualisation

The notebook shows:

- an eigenvalue and eigengap plot, which visually supports the choice $k = 4$,
- a cluster-size bar chart, which confirms the balanced partition,
- and a 2D scatter plot of the spectral embedding, where the four clusters appear clearly separated.

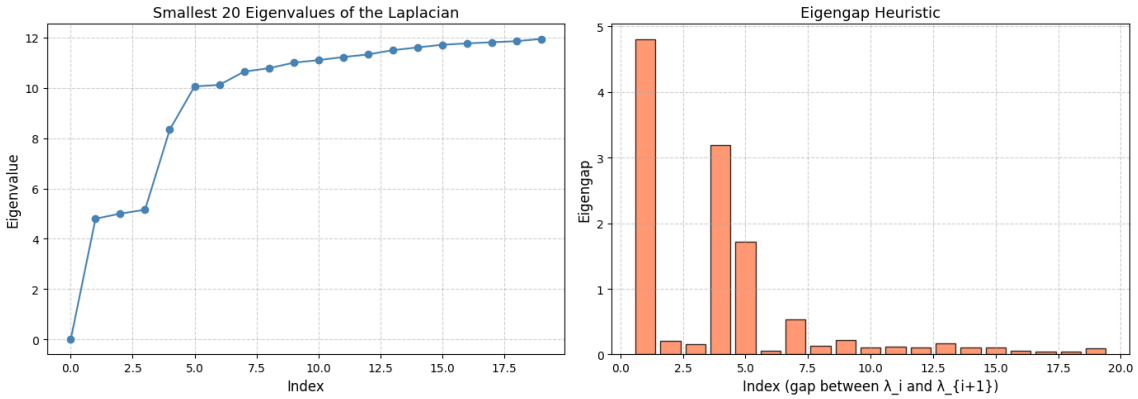

Figure 2: Smallest 20 eigenvalues of the Laplacian (left) and corresponding eigengaps (right). The largest non-trivial gap after λ_4 indicates $k = 4$.

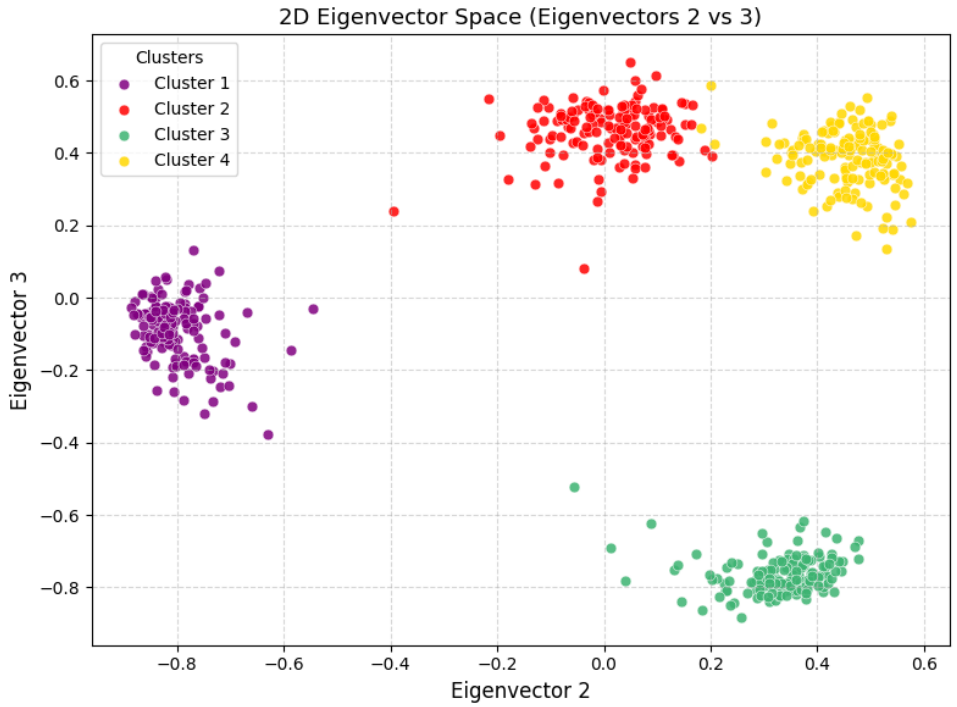

Figure 3: 2D visualization of the spectral embedding using selected eigenvectors. The four clusters are clearly separated in the embedding space. This plot is used only for visualization; clustering is performed using the full k-dimensional spectral embedding.

5 Discussion

5.1 Interpretation of the Results

The equal cluster sizes and the strong eigengap suggest that the graph has a clear planted community structure. Spectral clustering is well suited to this type of graph because the Laplacian eigenvectors separate the nodes into groups that are easy for K-Means to recover.

The low final inertia and fast convergence of K-Means indicate that the spectral embedding is compact and well-separated. This matches the strong gap in the Laplacian spectrum.

5.2 Why the Unnormalized Laplacian Works Here

The unnormalized Laplacian is a good choice for this dataset because the node degrees are not extremely skewed. When graphs have highly uneven degree distributions or strongly imbalanced cluster sizes, the normalized Laplacian is often preferable. For this graph, the unnormalized form was sufficient and gave a clear result.

5.3 Reproducibility

The results are reproducible because:

- the random seed is fixed with `np.random.seed(42)`,
- K-Means uses `random_state=42`,
- the eigenvalues are explicitly sorted before selecting k,
- and the full node-to-cluster output is saved to `cluster_labels.csv`.

6 Conclusion

We applied spectral clustering to the 600-node graph using the unnormalized Laplacian, the 20 smallest eigenvalues, and the eigengap heuristic. The largest non-trivial gap appears after the fourth eigenvalue, so the number of clusters is:

$$k = 4.$$

The final pipeline is:

1. build the graph and Laplacian,
2. compute the smallest eigenvalues and eigenvectors,
3. choose k by eigengap,
4. row-normalize the spectral embedding,
5. run K-Means,
6. and save the final 1-based labels.

The final clustering is balanced, reproducible, and consistent with the graph spectrum.

Appendix: Output Files

The notebook produces the following final output:

- `cluster_labels.csv` — node-to-cluster assignments for all 600 nodes.